

concisely, but in full, describe these rag concepts:

Indexing phase vs query phase

Chunking strategies (size, overlap, hierarchy)

Embedding selection tradeoffs

Retriever vs reranker

Context window management

Here's a **concise but complete** explanation of each RAG concept, focused on *what it is*, *why it matters*, and *key tradeoffs*.

Indexing phase vs query phase

- **Indexing phase:** Offline (or periodic) preparation step. Documents are cleaned, chunked, embedded, and stored in a vector index (plus metadata).
Goal: Make knowledge searchable efficiently.
 - **Query phase:** Online step at inference time. User query is embedded, relevant chunks are retrieved, optionally reranked, and injected into the LLM prompt.
Goal: Fetch the *right context* quickly and accurately.
-

Chunking strategies (size, overlap, hierarchy)

- **Size:** Small chunks improve retrieval precision; large chunks preserve context but risk dilution.
 - **Overlap:** Overlapping chunks reduce boundary information loss but increase index size and redundancy.
 - **Hierarchy:** Multi-level chunks (document → section → paragraph) allow coarse-to-fine retrieval and better structural grounding.
-

Embedding selection tradeoffs

- **Semantic vs lexical sensitivity:** Some embeddings favor meaning, others exact phrasing.
 - **Dimensionality:** Higher dimensions improve expressiveness but increase storage and latency.
 - **Domain specificity:** General embeddings are versatile; domain-tuned embeddings improve accuracy on specialized corpora.
 - **Cost & speed:** Stronger models cost more and embed slower.
-

Retriever vs reranker

- **Retriever:** Fast, approximate search over embeddings (e.g., top-k by cosine similarity).
Strength: Speed and scalability.
 - **Reranker:** Slower, more precise scoring (often cross-attention) applied to retrieved candidates.
Strength: Relevance accuracy.
 - **Pattern:** Retriever narrows → reranker refines.
-

Context window management

- **Selection:** Decide *what* and *how many* chunks to include.
 - **Ordering:** Arrange context to maximize coherence and relevance.
 - **Compression:** Summarize or filter retrieved text to fit token limits.
 - **Tradeoff:** More context reduces hallucination risk but increases cost and can confuse the model.
-

If you want, I can collapse this into a **one-page RAG cheat sheet**, or map these directly to **failure modes** (e.g., hallucination, over-retrieval).



give me a full code example with options for each component

```
python
```

```
"""
```

```
A full, modular RAG example with options for each component:
```

- 1) Indexing phase vs Query phase
- 2) Chunking strategies: size, overlap, hierarchy
- 3) Embedding selection tradeoffs: TF-IDF (fast/local) vs SentenceTransformers
- 4) Retriever vs Reranker: Vector retriever + (optional) reranking (BM25 or CrossEncoder)
- 5) Context window management: token budgeting + packing + optional compression

```
This is designed to run with minimal deps:
```

- Always works with: scikit-learn (for TF-IDF embeddings + vector search via sklearn.metrics.pairwise_distances)
- Optional (better): sentence-transformers for semantic embeddings
- Optional rerank: rank_bm25 OR sentence-transformers CrossEncoder

```
Install (optional):
```

```
pip install scikit-learn numpy
pip install sentence-transformers
pip install rank-bm25
```

```
"""
```

```
from __future__ import annotations
```

```
import math
```

```
import re
```

```
from dataclasses import dataclass
```

```
from typing import Callable, Dict, Iterable, List, Optional, Sequence, Tuple
```

```
import numpy as np
```

```
# -----
```

```
# Utilities
```

```
# -----
```

```

def simple_tokenize(text: str) -> List[str]:
    # Cheap tokenizer; good enough for chunking/BM25 budgets
    return re.findall(r"\w+|[\^w\s]", text.lower(), flags=re.UNICODE)

def approx_token_count(text: str) -> int:
    # Approx tokens by whitespace-ish. Replace with tiktoken if you want e
    return max(1, len(simple_tokenize(text)))

def normalize_rows(x: np.ndarray) -> np.ndarray:
    denom = np.linalg.norm(x, axis=1, keepdims=True) + 1e-12
    return x / denom

# -----
# Chunking options
# -----

@dataclass
class Chunk:
    doc_id: str
    chunk_id: str
    text: str
    meta: Dict[str, str]

class Chunker:
    def chunk(self, doc_id: str, text: str) -> List[Chunk]:
        raise NotImplementedError

class FixedWindowChunker(Chunker):
    """
    size: number of tokens per chunk
    overlap: number of tokens to overlap between chunks
    """
    def __init__(self, size: int = 200, overlap: int = 40):
        assert size > 0
        assert 0 <= overlap < size
        self.size = size
        self.overlap = overlap

```

```
def chunk(self, doc_id: str, text: str) -> List[Chunk]:
    toks = simple_tokenize(text)
    chunks: List[Chunk] = []
    step = self.size - self.overlap
    for i, start in enumerate(range(0, len(toks), step)):
        window = toks[start : start + self.size]
        if not window:
            break
        chunk_text = " ".join(window)
        chunks.append(
            Chunk(
                doc_id=doc_id,
                chunk_id=f"{doc_id}::w{i}",
                text=chunk_text,
                meta={"strategy": "fixed_window", "start_tok": str(start)
            )
        )
        if start + self.size >= len(toks):
            break
    return chunks
```

 Copy code

```
class HierarchicalChunker(Chunker):
    """
    Hierarchy: doc -> section -> subchunks

    - Splits on headings like:
        "## Heading" or lines in ALL CAPS ending with ":".
    - Then optionally subchunks each section using FixedWindowChunker.
    """
    def __init__(self, section_window: Optional[FixedWindowChunker] = None):
        self.section_window = section_window

    def _split_sections(self, text: str) -> List[Tuple[str, str]]:
        lines = text.splitlines()
        sections: List[Tuple[str, List[str]]] = []
        cur_title = "INTRO"
        cur_buf: List[str] = []
        heading_re = re.compile(r"^\s*#{1,6}\s+(.+?)\s*$")
        caps_re = re.compile(r"^[A-Z0-9][A-Z0-9 \-]{3,}:\s*$")
```

```

def flush():
    nonlocal cur_title, cur_buf
    if cur_buf:
        sections.append((cur_title, cur_buf))
    cur_buf = []

for line in lines:
    m = heading_re.match(line)
    if m:
        flush()
        cur_title = m.group(1).strip()
        continue
    if caps_re.match(line):
        flush()
        cur_title = line.strip().rstrip(":")
        continue
    cur_buf.append(line)

```

≡ ChatGPT 5.2 ▾

↑ ...

```

out: List[Tuple[str, str]] = []
for title, buf in sections:
    body = "\n".join(buf).strip()
    if body:
        out.append((title, body))
return out

```

```

def chunk(self, doc_id: str, text: str) -> List[Chunk]:
    sections = self._split_sections(text)
    chunks: List[Chunk] = []
    for si, (title, body) in enumerate(sections):
        sec_id = f"{doc_id}::sec{si}"
        sec_meta = {"strategy": "hierarchical", "section": title}
        # Section-level chunk (coarse)
        chunks.append(
            Chunk(doc_id=doc_id, chunk_id=sec_id, text=body, meta=sec_
        )
        # Optional finer subchunks
        if self.section_window:
            subs = self.section_window.chunk(doc_id=sec_id, text=body)

```

```

        for sub in subs:
            sub.meta = sub.meta | sec_meta | {"level": "subchunk"}
            chunks.append(sub)
        return chunks

# -----
# Embeddings options
# -----

class Embedder:
    def fit(self, texts: Sequence[str]) -> None:
        """Optional: some embedders (TF-IDF) need fitting."""
        return

    def encode(self, texts: Sequence[str]) -> np.ndarray:
        raise NotImplementedError

class TfidfEmbedder(Embedder):
    """Fast + local + lexical-ish; great baseline for small/medium corpora"""
    def __init__(self):
        from sklearn.feature_extraction.text import TfidfVectorizer
        self.vectorizer = TfidfVectorizer(
            lowercase=True,
            ngram_range=(1, 2),
            max_features=200_000,
        )

    def fit(self, texts: Sequence[str]) -> None:
        self.vectorizer.fit(texts)

    def encode(self, texts: Sequence[str]) -> np.ndarray:
        X = self.vectorizer.transform(texts)
        return X.astype(np.float32).toarray()

class SentenceTransformerEmbedder(Embedder):
    """Semantic embeddings (better recall), needs sentence-transformers."""
    def __init__(self, model_name: str = "sentence-transformers/all-MiniLM
        from sentence_transformers import SentenceTransformer
        self.model = SentenceTransformer(model_name)

```

```

def encode(self, texts: Sequence[str]) -> np.ndarray:
    emb = self.model.encode(list(texts), normalize_embeddings=True)
    return np.asarray(emb, dtype=np.float32)

# -----
# Retrieval + reranking options
# -----

@dataclass
class ScoredChunk:
    chunk: Chunk
    score: float

class VectorRetriever:
    """Cosine similarity over embeddings (works for both TF-IDF & ST, but !
    def __init__(self, embeddings: np.ndarray, chunks: List[Chunk], normal:
        self.chunks = chunks
        self.emb = embeddings.astype(np.float32)
        self.normalize = normalize
        if normalize:
            self.emb = normalize_rows(self.emb)

    def retrieve(self, query_emb: np.ndarray, top_k: int = 8) -> List[Scored
        q = query_emb.astype(np.float32)
        if q.ndim == 1:
            q = q[None, :]
        if self.normalize:
            q = normalize_rows(q)
        # cosine similarity: dot if normalized
        sims = (self.emb @ q.T).reshape(-1)
        idx = np.argsort(-sims)[:top_k]
        return [ScoredChunk(self.chunks[i], float(sims[i])) for i in idx]

class Reranker:
    def rerank(self, query: str, candidates: List[ScoredChunk], top_k: int
        raise NotImplementedError

class NoReranker(Reranker):

```



```

def rerank(self, query: str, candidates: List[ScoredChunk], top_k: int
    return candidates[:top_k]

class BM25Reranker(Reranker):
    """Lexical reranking: good precision. Needs rank_bm25."""
    def __init__(self):
        from rank_bm25 import BM25Okapi
        self.BM25Okapi = BM25Okapi

    def rerank(self, query: str, candidates: List[ScoredChunk], top_k: int
        docs = [simple_tokenize(c.chunk.text) for c in candidates]
        bm25 = self.BM25Okapi(docs)
        q = simple_tokenize(query)
        scores = bm25.get_scores(q)
        order = np.argsort(-scores)[:top_k]
        return [ScoredChunk(candidates[i].chunk, float(scores[i])) for i in

class CrossEncoderReranker(Reranker):
    """Best precision (slower). Needs sentence-transformers CrossEncoder."""
    def __init__(self, model_name: str = "cross-encoder/ms-marco-MiniLM-L-6
        from sentence_transformers import CrossEncoder
        self.model = CrossEncoder(model_name)

    def rerank(self, query: str, candidates: List[ScoredChunk], top_k: int
        pairs = [(query, c.chunk.text) for c in candidates]
        scores = self.model.predict(pairs)
        order = np.argsort(-scores)[:top_k]
        return [ScoredChunk(candidates[i].chunk, float(scores[i])) for i in

# -----
# Context window management options
# -----

@dataclass
class ContextConfig:
    max_context_tokens: int = 800
    per_chunk_max_tokens: int = 250
    dedupe: bool = True
    # Compression: if True, truncate each chunk to per_chunk_max_tokens

```

```
compress: bool = True
# Ordering: "score" or "original"
order: str = "score"

def pack_context(chunks: List[ScoredChunk], cfg: ContextConfig) -> Tuple[List[ScoredChunk], List[Chunk]]:
    chosen: List[Chunk] = []
    seen = set()
    total = 0

    ordered = chunks
    if cfg.order == "score":
        ordered = sorted(chunks, key=lambda x: x.score, reverse=True)

    for sc in ordered:
        key = sc.chunk.chunk_id
        if cfg.dedupe and key in seen:
            continue
        text = sc.chunk.text
        if cfg.compress:
            toks = simple_tokenize(text)
            toks = toks[: cfg.per_chunk_max_tokens]
            text = " ".join(toks)

        t = approx_token_count(text)
        if total + t > cfg.max_context_tokens:
            continue

        # keep a copy with compressed text if needed
        kept = Chunk(
            doc_id=sc.chunk.doc_id,
            chunk_id=sc.chunk.chunk_id,
            text=text,
            meta=sc.chunk.meta,
        )
        chosen.append(kept)
        seen.add(key)
        total += t

    if total >= cfg.max_context_tokens:
        break
```

```

context = "\n\n---\n\n".join(
    [f"[{c.doc_id} | {c.chunk_id} | {c.meta.get('section', '')}] \n{c.t"
)
return context, chosen

# -----
# RAG System (index + query)
# -----

@dataclass
class RagConfig:
    # Chunking
    chunking: str = "fixed"          # "fixed" or "hierarchical"
    chunk_size: int = 220
    chunk_overlap: int = 50
    # Embeddings
    embedding: str = "tfidf"         # "tfidf" or "st"
    st_model: str = "sentence-transformers/all-MiniLM-L6-v2"
    # Retrieval
    retrieve_top_k: int = 12
    # Reranking
    reranker: str = "none"          # "none" or "bm25" or "cross"
    rerank_top_k: int = 6
    cross_model: str = "cross-encoder/ms-marco-MiniLM-L-6-v2"
    # Context management
    context: ContextConfig = ContextConfig()

class RagSystem:
    def __init__(self, cfg: RagConfig):
        self.cfg = cfg
        self.chunker = self._build_chunker(cfg)
        self.embedder = self._build_embedder(cfg)
        self.reranker = self._build_reranker(cfg)

        self.chunks: List[Chunk] = []
        self.emb: Optional[np.ndarray] = None
        self.retriever: Optional[VectorRetriever] = None

```

```

# ----- builders -----
def _build_chunker(self, cfg: RagConfig) -> Chunker:
    if cfg.chunking == "fixed":
        return FixedWindowChunker(size=cfg.chunk_size, overlap=cfg.chunk_overlap)
    if cfg.chunking == "hierarchical":
        return HierarchicalChunker(
            section_window=FixedWindowChunker(size=cfg.chunk_size, overlap=cfg.chunk_overlap)
        )
    raise ValueError(f"Unknown chunking={cfg.chunking}")

def _build_embedder(self, cfg: RagConfig) -> Embedder:
    if cfg.embedding == "tfidf":
        return TfIdfEmbedder()
    if cfg.embedding == "st":
        return SentenceTransformerEmbedder(cfg.st_model)
    raise ValueError(f"Unknown embedding={cfg.embedding}")

def _build_reranker(self, cfg: RagConfig) -> Reranker:
    if cfg.reranker == "none":
        return NoReranker()
    if cfg.reranker == "bm25":
        return BM25Reranker()
    if cfg.reranker == "cross":
        return CrossEncoderReranker(cfg.cross_model)
    raise ValueError(f"Unknown reranker={cfg.reranker}")

# ----- Indexing phase -----
def index(self, documents: Dict[str, str]) -> None:
    """
    Indexing phase:
    docs -> chunk -> embed -> store (chunks + embeddings) -> build retriever
    """
    all_chunks: List[Chunk] = []
    for doc_id, text in documents.items():
        all_chunks.extend(self.chunker.chunk(doc_id, text))

    self.chunks = all_chunks
    corpus = [c.text for c in self.chunks]

    # Fit if needed (TF-IDF)

```

```

self.embedder.fit(corpus)

self.emb = self.embedder.encode(corpus)
self.retriever = VectorRetriever(self.emb, self.chunks, normalize=

# ----- Query phase -----
def query(self, question: str) -> Dict[str, object]:
    """
    Query phase:
        q -> embed -> retrieve topK -> rerank -> context pack -> return |
    """
    if not self.retriever:
        raise RuntimeError("Call index() first.")

    q_emb = self.embedder.encode([question])[0]
    retrieved = self.retriever.retrieve(q_emb, top_k=self.cfg.retrieve_
    reranked = self.reranker.rerank(question, retrieved, top_k=self.cfg

    context_text, chosen_chunks = pack_context(reranked, self.cfg.cont

    # Your LLM call would use:
    # prompt = f"Answer using ONLY the context.\n\nContext:\n{context
    return {
        "question": question,
        "retrieved": retrieved,
        "reranked": reranked,
        "context": context_text,
        "chosen_chunks": chosen_chunks,
    }

# -----
# Demo main + "tests"
# -----

def _demo_docs() -> Dict[str, str]:
    return {
        "doc1": """## Indexing vs Query
Indexing is the offline step: parse docs, chunk them, embed them, store vec
Querying is online: embed the question, retrieve nearest chunks, optionally

```

Chunking

Chunk size trades off precision vs context. Overlap helps avoid boundary loss

"doc2": ""RETRIEVER VS RERANKER:

A retriever is fast and approximate (vector similarity). A reranker is slow and precise (cross-attention). Often using cross-attention between query and chunk. Common pattern: retriever → reranker

CONTEXT WINDOW MANAGEMENT:

You must fit within the model's token limit. Strategies: top-k packing, deduplication, and compressing chunks into summaries.""",

"doc3": ""EMBEDDING SELECTION:

TF-IDF is lexical and fast; semantic embedders generalize better. Higher dimensional embeddings can boost retrieval quality on specialized corpora

}

```
def run_demo(cfg: RagConfig) -> None:
```

```
    rag = RagSystem(cfg)
```

```
    rag.index(_demo_docs())
```

```
    out = rag.query("How are retriever and reranker different, and what is the difference?")
```

```
    print("\n=== CONTEXT (packed) ===")
```

```
    print(out["context"][:1200], "... \n")
```

```
    print("=== TOP RERANKED CHUNKS ===")
```

```
    for i, sc in enumerate(out["reranked"], 1):
```

```
        print(f"{i:02d}. score={sc.score:.4f} id={sc.chunk.chunk_id} meta={sc.chunk.metadata}")
```

```
def main():
```

```
    # ---- OPTION SETS (toggle these) ----
```

```
    # 1) Chunking options: "fixed" vs "hierarchical"
```

```
    # 2) Embedding options: "tfidf" (baseline) vs "st" (semantic)
```

```
    # 3) Reranking options: "none" vs "bm25" vs "cross"
```

```
    # 4) Context management in cfg.context
```

```
    cfg = RagConfig(
```

```
        chunking="hierarchical",
```

```
        chunk_size=90,
```

```
        chunk_overlap=20,
```

```
        embedding="tfidf",
```

```
        # change to "st" if you installed sentence-transformers
```

```
        reranker="none",
```

```
        # try: "bm25" or "cross"
```

```

retrieve_top_k=10,
rerank_top_k=5,
context=ContextConfig(
    max_context_tokens=220,
    per_chunk_max_tokens=90,
    dedupe=True,
    compress=True,
    order="score",
),
)

run_demo(cfg)

# --- "tests" / sanity checks ---
rag = RagSystem(cfg)
rag.index(_demo_docs())
out = rag.query("What is indexing in RAG?")
assert "offline" in out["context"].lower() or "indexing" in out["context"]
assert len(out["retrieved"]) <= cfg.retrieve_top_k
assert len(out["reranked"]) <= cfg.rerank_top_k
print("\nSanity checks passed.")

if __name__ == "__main__":
    main()

```

How to use this quickly

- Start with `embedding="tfidf"` and `reranker="none"` (fast, zero extra deps beyond scikit-learn).
- Upgrade in-place:
 - Better recall: set `embedding="st"` (install sentence-transformers).
 - Better precision: set `reranker="bm25"` or `reranker="cross"`.

If you tell me what LLM you're calling (OpenAI / local / HF), I'll plug in the final "generate answer" call and make the prompt + citations format clean.



@dataclass

@dataclass is a **Python decorator** that automatically generates boilerplate code for simple data-holding classes.

What it gives you (for free)

When you write:

python

 Copy code

```
from dataclasses import dataclass

@dataclass
class Chunk:
    doc_id: str
    chunk_id: str
    text: str
    meta: dict
```

Python **automatically creates**:

- `__init__(...)`
- `__repr__()` (nice printable form)
- `__eq__(...)` (value-based equality)

So this:

python

 Copy code

```
c = Chunk("doc1", "c1", "hello", {"section": "intro"})
```

works **without** writing an `__init__` yourself.

Why it's used (especially in ML / RAG code)

Dataclasses are ideal when a class is:

- Mostly **data**, not behavior

- Passed around between components
- Needs to be **explicit, typed, and readable**

That's why they're perfect for:

- Chunks
- Config objects
- Scored results
- Pipeline outputs

Equivalent code *without* @dataclass

python

 Copy code

```
class Chunk:
    def __init__(self, doc_id, chunk_id, text, meta):
        self.doc_id = doc_id
        self.chunk_id = chunk_id
        self.text = text
        self.meta = meta

    def __repr__(self):
        return f"Chunk(doc_id={self.doc_id}, chunk_id={self.chunk_id})"
```

@dataclass removes all that noise.

Common useful options

python

 Copy code

```
@dataclass(frozen=True)    # immutable (hashable)
class Point:
    x: float
    y: float
```

python

 Copy code

```
@dataclass(order=True)      # adds <, > comparisons
class ScoredChunk:
    score: float
    chunk: Chunk
```

python

 Copy code

```
from dataclasses import field
```

```
@dataclass
class Config:
    top_k: int = 5
    tags: list = field(default_factory=list) # avoid mutable defaults
```

Why it mattered in the RAG code you saw

- **Clear contracts** between stages (chunk → retrieval → reranking)
- **Config objects** you can tweak without changing logic
- **Type safety** for large pipelines

If you want, I can show:

- how `@dataclass` compares to `NamedTuple`
- when **not** to use dataclasses
- or how this maps to **production ML code patterns**

     ...

naming convention `_functions`

Here's the **clean, Pythonic naming convention for functions**, with the meanings of `_` spelled out clearly.

1. snake_case (standard for functions)

Always use lowercase with underscores.

python

 Copy code

```
def build_index():  
    pass  
  
def retrieve_top_k():  
    pass
```

Why: PEP 8 standard → readable, consistent, expected in interviews and production.

2. Leading underscore: `_function`

Means “internal use” (non-public API).

python

 Copy code

```
def _tokenize(text):  
    pass
```

- Not imported with `from module import *`
 - Signals: “*don’t call this outside this module*”
 - Very common in libraries and ML pipelines
-

3. Trailing underscore: `function_`

Avoids naming conflicts with Python keywords or built-ins.

python

 Copy code

```
def filter_(items):  
    pass  
  
def class_(name):  
    pass
```

Use only when needed.

4. Double leading underscore: `__function`

Name mangling (classes only).

python

 Copy code

```
class Retriever:
    def __score(self):
        pass
```

Becomes `_Retriever__score` internally.

- Prevents accidental overrides in subclasses
- Rarely needed in ML / RAG code

5. Double underscores on both sides: `__function__`

Reserved for Python “dunder” methods.

python

 Copy code

```
def __init__(self):
    pass

def __call__(self, x):
    pass
```

 Never invent your own dunder names.

6. Single underscore `_` (throwaway / convention)

python

 Copy code

```
for _ in range(3):
    pass
```

- Means “value intentionally unused”
- Also used for:
 - Last REPL result

- Placeholder arguments

7. Module-level helper naming (very common in ML code)

python

 Copy code

```
def _build_chunker():  
    pass  
  
def _load_model():  
    pass
```

Pattern:

- Public API → no underscore
- Helpers → leading underscore

8. Recommended pattern for pipelines (RAG / ML)

python

 Copy code

```
def index_documents():  
    pass  
  
def retrieve_candidates():  
    pass  
  
def rerank_candidates():  
    pass  
  
def pack_context():  
    pass
```

Helpers:

python

 Copy code

```
def _normalize_embeddings():  
    pass
```

```
def _count_tokens():  
    pass
```

9. Summary table

Pattern	Meaning	When to use
snake_case	Standard	Always
_func	Internal	Helper functions
func_	Keyword conflict	Rare
__func	Name mangling	Advanced OOP
__func__	Python internals	Never define yourself
_	Throwaway	Loops, unused vars

TL;DR (interview-safe answer)

- Use snake_case for functions.
- Prefix with _ for internal helpers.
- Avoid custom __dunder__ names.

If you want, I can give you a **style guide specifically for ML / RAG / research codebases** (what reviewers actually expect).

